

间,但如果数据更新(增、删、改)频繁,系统会花费许多时间来维护索引,从而降低了查询效率。这时可以删除一些不必要的索引。

在 SQL 中,删除索引使用 DROP INDEX 语句,其一般格式如下:

DROP INDEX <索引名>;

[例 3.15] 删除 Student 表的 Idx_StuSname 索引。

DROP INDEX Idx_StuSname;

删除索引时,系统会同时从数据字典中删去有关该索引的描述。

3.2.4 数据字典

数据字典是关系数据库管理系统内部的一组系统表,它记录了数据库中所有的定义信息,包括关系模式定义、视图定义、索引定义、完整性约束定义、各类用户对数据库的操作权限、统计信息等。关系数据库管理系统在执行 SQL 的数据定义语句时,实际上就是把这些定义信息存入系统的数据字典以更新其中的相应信息。在进行查询处理和查询优化时,关系数据库管理系统要根据数据字典中的信息执行处理算法和优化算法,因此数据字典是关系数据库管理系统运行的重要依据。

3.3 数据查询

数据查询是数据库的核心操作。SQL 提供了 SELECT 语句进行数据查询,该语句具有灵活的使用方式和丰富的功能。

SELECT 语句的一般格式如下:

```
SELECT[ ALL|DISTINCT]<目标列表达式>[别名][,<目标列表达式>[别名]]...  
FROM <表名或视图名>[别名][,<表名或视图名>[别名]]... | (<SELECT 语句>)[AS]<别名>  
[ WHERE <条件表达式>]  
[ GROUP BY <列名 1>[ HAVING <条件表达式>]]  
[ ORDER BY <列名 2>[ ASC|DESC]]  
[ LIMIT <行数 1>[ OFFSET <行数 2>]];
```

SELECT 语句的含义是,根据 WHERE 子句的条件表达式从 FROM 子句指定的基本表、视图或派生表中找出满足条件的元组,再按 SELECT 子句中的目标列表达式选出元组中的属性值形成结果表。

如果有 GROUP BY 子句,则将结果按<列名 1>的值进行分组,该属性列值相等的元组为一个组,通常会在每组中作用聚集函数。如果 GROUP BY 子句带 HAVING 短语,则只有满足指定条件的组才予以输出。

如果有 ORDER BY 子句, 则结果表还要按<列名 2>的值的升序或降序排序。

如果有 LIMIT 子句, 则限制 SELECT 语句查询结果的数量为<行数 1>行, OFFSET <行数 2>表示在计算<行数 1>行前忽略<行数 2>行。OFFSET 子句可省略, 代表不忽略任何行。LIMIT 是和 GROUP BY、ORDER BY 并列的子句。

SELECT 语句既可以完成简单的单表查询, 也可以完成复杂的连接查询和嵌套查询。下面以“学生选课”数据库为例, 说明 SELECT 语句的各种用法。

3.3.1 单表查询

单表查询是指仅涉及一个表的查询。

1. 选择表中的若干列

选择表中的全部或部分列为关系代数的投影运算。

(1) 查询指定列

在很多情况下, 用户只对表中的一部分属性列感兴趣, 这时可以通过在 SELECT 子句的<目标列表表达式>中指定要查询的属性列。

[例 3.16] 查询全体学生的学号与姓名。

```
SELECT Sno,Sname FROM Student;
```

该语句的执行过程可以是这样的: 从 Student 表中取出一个元组, 取出该元组在属性 Sno 和 Sname 上的值, 形成一个新的元组作为输出。对 Student 表中的所有元组做相同的处理, 最后形成一个结果关系作为输出。

[例 3.17] 查询全体学生的姓名、学号、主修专业。

```
SELECT Sname,Sno,Smajor FROM Student;
```

<目标列表表达式>中各个列的先后顺序可以与表中的顺序不一致。用户可以根据应用的需要改变列的显示顺序。本例中先列出姓名, 再列出学号和主修专业。

(2) 查询全部列

将表中的所有属性列都选出来有两种方法, 一种方法就是在 SELECT 关键字后列出所有列名; 如果列的显示顺序与其在基表中的顺序相同, 也可以简单地将<目标列表表达式>指定为“*”。

[例 3.18] 查询全体学生的详细记录。

```
SELECT * FROM Student;
```

等价于

```
SELECT Sno,Sname,Ssex,Sbirthdate,Smajor FROM Student;
```

(3) 查询经过计算的值

SELECT 语句的<目标列表表达式>可以是算术表达式、字符串常量、函数等。

[例 3.19] 查询全体学生的姓名及其年龄。

查询要求中“年龄”不能在学生表中直接查到,需要通过计算得到。

如果使用 KingbaseES 数据库系统,其 SQL 语句如下:

```
SELECT Sname, (extract(year from current_date)-extract(year from Sbirthdate)) "年龄"
FROM Student; /* SELECT 语句的第 2 列是表达式,起了别名“年龄” */
```

extract(year from current_date) 是 Kingbase 提供的内置函数,其功能是获取当前日期的年份,extract(year from Sbirthdate) 为获取表中出生日期的年份。

学生的出生年份是从属性列“出生日期”中计算得到的,不同的数据库管理系统提供的内置函数不同,语法也不尽相同。所以,读者在书写 SQL 语句时一定要参考所使用的产品手册。本书对部分数据库管理系统产品的主要函数进行了整理,放在第 8 章的二维码 8.1 中,供读者参考。

本例查询结果如图 3.3 所示。

注意: CURRENT_DATE 为实际上机实验的日期,读者的实际操作结果与这里的输出结果会有出入。此外,本例计算年龄仅考虑年份,如果以具体出生日期为界限的话就可能有误差了。本章后面的示例也均作此考虑。

用户还可以通过指定别名来改变查询结果的列标题,这对于含算术表达式、常量、函数名的目标列表达式尤为有用。在例 3.19 中就定义了“年龄”别名。

用户也可以在 SELECT 语句中加上说明含义的一个列标题。例如在例 3.20 中加上'Date of Birth:',表明右边一列是出生日期。

[例 3.20] 查询全体学生的姓名、出生日期和主修专业。

```
SELECT Sname,'Date of Birth:',Sbirthdate,Smajor
FROM Student;
```

查询结果如图 3.4 所示。

Sname	年龄
李勇	21
刘晨	22
王敏	20
张立	21
陈新奇	20
赵明	21
王佳佳	20

图 3.3 例 3.19 查询结果

Sname	Date of Birth:	Sbirthdate	Smajor
李勇	Date of Birth:	2000-3-8	信息安全
刘晨	Date of Birth:	1999-9-1	计算机科学与技术
王敏	Date of Birth:	2001-8-1	计算机科学与技术
张立	Date of Birth:	2000-1-8	计算机科学与技术
陈新奇	Date of Birth:	2001-11-1	信息管理与信息系统
赵明	Date of Birth:	2000-6-12	数据科学与大数据技术
王佳佳	Date of Birth:	2001-12-7	数据科学与大数据技术

图 3.4 例 3.20 查询结果

2. 选择表中的若干元组

(1) 消除取值重复的行

两个本来并不完全相同的元组在投影到指定的某些列上后,可能会变成相同的行。可以用 DISTINCT 消除它们。

[例 3.21] 查询选修了课程的学生学号。

```
SELECT Sno
FROM SC;
```

该查询结果里包含了许多重复的行。如想去掉结果表中的重复行,必须指定 DISTINCT:

```
SELECT DISTINCT Sno
FROM SC;
```

查询结果如图 3.5 所示。

如果没有指定 DISTINCT 关键词,则默认值为 ALL,即保留结果表中取值重复的行。

```
SELECT Sno
FROM SC;
```

等价于

```
SELECT ALL Sno
FROM SC;
```

Sno
20180001
20180002
20180003
20180004
20180005

图 3.5 例 3.21 查询结果

(2) 查询满足条件的元组

查询满足指定条件的元组可以通过 WHERE 子句实现。WHERE 子句常用的查询条件如表 3.5 所示。

表 3.5 常用的查询条件

查询条件	谓词
比较	=, >, <, >=, <=, !=, <>, !>, !<; NOT+上述比较运算符
确定范围	BETWEEN AND, NOT BETWEEN AND
确定集合	IN, NOT IN
字符匹配	LIKE, NOT LIKE
空值	IS NULL, IS NOT NULL
多重条件(逻辑运算)	AND, OR, NOT

① 比较大小

用于进行比较的运算符一般包括=(等于), >(大于), <(小于), >=(大于或等于), <=

(小于或等于), !=或<>(不等于), !>(不大于), !<(不小于)。

[例 3.22] 查询主修计算机科学与技术专业全体学生的姓名。

```
SELECT Sname
FROM Student
WHERE Smajor='计算机科学与技术'; /* 字符串常数要用单引号(英文符号)括起来 */
```

关系数据库管理系统执行该查询的一种可能过程是：对 Student 表进行全表扫描，取出一个元组，检查该元组在 Smajor 列的值是否等于“计算机科学与技术”，如果相等，则取出 Sname 列的值形成一个新的元组输出；否则跳过该元组，取下一个元组。重复该过程，直到处理完 Student 表的所有元组。

如果全校有数万个学生，主修计算机科学与技术专业的学生人数约是全校学生的 9%，则可以在 Student 表的 Smajor 列上建立索引，系统会利用该索引找出 Smajor='计算机科学与技术'的元组，从中取出 Sname 列值形成结果关系。这就避免了对 Student 表的全表扫描，加快了查询速度。

注意：如果全校学生较少，索引查找不一定能提高查询效率，系统仍会使用全表扫描。这由查询优化器按照算法规则或估计执行代价来做出选择。

[例 3.23] 查询 2000 年及 2000 年后出生的所有学生的姓名及其性别。

```
SELECT Sname, Ssex
FROM Student
WHERE extract(year from Sbirthdate) >= 2000;
/* 函数 extract(year from Sbirthdate) 用于从出生日期中抽取年份 */
```

[例 3.24] 查询考试成绩不及格的学生的学号。

```
SELECT DISTINCT Sno
FROM SC
WHERE Grade<60;
```

这里使用了 DISTINCT 短语，当一个学生有多门课程不及格时，其学号也只列一次。

② 确定范围

谓词 BETWEEN...AND... 和 NOT BETWEEN...AND... 可以用来查找属性值在(或不在)指定范围内的元组，其中 BETWEEN 后是范围的下限(即低值)，AND 后是范围的上限(即高值)。

[例 3.25] 查询年龄在 20~23 岁(包括 20 岁和 23 岁)之间的学生的姓名、出生日期和主修专业。

```
SELECT Sname, Sbirthdate, Smajor
FROM Student
```

```
WHERE extract(year from current_date) - extract(year from Sbirthdate)
      BETWEEN 20 AND 23;
```

[例 3.26] 查询年龄不在 20~23 岁范围内的学生的姓名、出生日期和主修专业。

```
SELECT Sname,Sbirthdate,Smajor
FROM Student
WHERE extract(year from current_date) - extract(year from Sbirthdate)
      NOT BETWEEN 20 AND 23;
```

③ 确定集合

谓词 IN 可以用来查找属性值属于指定集合的元组。

[例 3.27] 查询计算机科学与技术专业和信息安全专业的学生的姓名及性别。

```
SELECT Sname,Ssex
FROM Student
WHERE Smajor IN ( '计算机科学与技术','信息安全' );
```

与 IN 相对的谓词是 NOT IN，用于查找属性值不属于指定集合的元组。

[例 3.28] 查询非计算机科学与技术专业和信息安全专业的学生的姓名和性别。

```
SELECT Sname,Ssex
FROM Student
WHERE Smajor NOT IN ( '计算机科学与技术','信息安全' );
```

④ 字符匹配

谓词 LIKE 可以用来进行字符串的匹配。其一般语法格式如下：

[NOT]LIKE '<匹配串>' [ESCAPE '<换码字符>']

即查找指定的属性列值与<匹配串>相匹配的元组。<匹配串>可以是一个完整的字符串，也可以含有通配符%和_。其中：

a. %(百分号)代表任意长度(长度可以为0)的字符串。例如“a%b”表示以a开头，以b结尾的任意长度的字符串。如acb、addgb、ab等都满足该匹配串。

b. _(下划线)代表任意单个字符。例如“a_b”表示以a开头，以b结尾的长度为3的任意字符串，如acb、afb等都满足该匹配串。

[例 3.29] 查询学号为 20180003 的学生的详细情况。

```
SELECT *
FROM Student
WHERE Sno LIKE '20180003';
```

等价于

```
SELECT *  
FROM Student  
WHERE Sno='20180003';
```

如果 LIKE 后面的匹配串中不含通配符,则可以用“=”(等于)运算符取代 LIKE 谓词,用“!=”或“<>”(不等于)运算符取代 NOT LIKE 谓词。

[例 3.30] 查询所有姓刘的学生的姓名、学号和性别。

```
SELECT Sname,Sno,Ssex  
FROM Student  
WHERE Sname LIKE '刘%';
```

[例 3.31] 查询 2018 级学生的学号和姓名。

```
SELECT Sno,Sname  
FROM Student  
WHERE Sno LIKE '2018%';      /* 学号的数据类型是字符,用字符匹配 */
```

[例 3.32] 查询课程号为 81 开头,最后一位是 6 的课程名称和课程号。

```
SELECT Cname,Cno  
FROM Course  
WHERE Cno LIKE '81__6';      /* 注意课程关系中课程号为固定长度,占 5 个字符大小 */
```

[例 3.33] 查询所有不姓刘的学生的姓名、学号和性别。

```
SELECT Sname,Sno,Ssex  
FROM Student  
WHERE Sname NOT LIKE '刘%';
```

如果用户要查询的字符串本身就含有通配符%或_,这时就要使用 ESCAPE '<换码字符>'短语对通配符进行转义了。

[例 3.34] 查询 DB_Design 课程的课程号和学分。

```
/* 假设已插入了一条数据库设计课程元组,课程名称为 DB_Design */  
SELECT Cno,Ccredit  
FROM Course  
WHERE Cname LIKE 'DB\_Design' ESCAPE '\';
```

“ESCAPE ‘\’”表示“\”为换码字符。这样匹配串中紧跟在“\”后面的字符“_”不再具有通配符的含义,转义为普通的“_”字符。

[例 3.35] 查询以“DB_”开头,且倒数第三个字符为 i 的课程的具体情况。

```
SELECT *
```

```
FROM Course
WHERE Cname LIKE 'DB\__%i\_ ' ESCAPE '\';
```

这里的匹配串为'DB__%i_ ' ESCAPE '\'. 第一个“_”前有换码字符“\”，所以它被转义为普通的“_”字符。而“i”后的两个“_”的前面均没有换码字符“\”，所以它们仍作为通配符。

⑤ 涉及空值的查询

[例 3.36] 某些学生选修课程后没有参加考试，所以有选课记录但没有考试成绩。查询缺少成绩的学生的学号和相应的课程号。

```
SELECT Sno,Cno
FROM SC
WHERE Grade IS NULL;          /* 分数 Grade 是空值 */
```

注意这里的“IS”不能用等号(=)代替。

[例 3.37] 查所有有成绩的学生的学号和选修的课程号。

```
SELECT Sno,Cno
FROM SC
WHERE Grade IS NOT NULL;
```

⑥ 多重条件查询

逻辑运算符 AND 和 OR 可用来连接多个查询条件。AND 的优先级高于 OR，但用户可以用括号改变优先级。

[例 3.38] 查询主修计算机科学与技术专业，在 2000 年(包括 2000 年)以后出生的学生的学号、姓名和性别。

```
SELECT Sno,Sname,Ssex
FROM Student
WHERE Smajor='计算机科学与技术' AND extract(year from Sbirthdate)>=2000;
```

在例 3.27 中的 IN 谓词实际上是多个 OR 运算符的缩写，因此该例中的查询也可以用 OR 运算符写成如下等价形式：

```
SELECT Sname,Ssex
FROM Student
WHERE Smajor='计算机科学与技术' OR Smajor='信息安全';
```

3. ORDER BY 子句

用户可以用 ORDER BY 子句对查询结果按照一个或多个属性列的升序(ASC)或降序(DESC)排列，默认值为升序。

[例 3.39] 查询选修了 81003 号课程的学生的学号及其成绩，查询结果按分数的降序排列。


```
SELECT Sno,Grade
FROM SC
WHERE Cno='81003'
ORDER BY Grade DESC;
```

对于空值,排序时显示的次序由具体系统实现来决定。例如按升序排,含空值的元组最后显示;按降序排,含空值的元组则最先显示。各个系统的实现可以不同,请读者参考产品手册。

[例 3.40] 查询全体学生选修课程情况,查询结果先按照课程号升序排列,同一课程中按成绩降序排列。

```
SELECT *
FROM SC
ORDER BY Cno,Grade DESC;

/* 排序字段的说明默认为升序 ASC,可以省略,而 DESC 需要明确写出来。 */
```

注意: 这里是对查询结果进行排序,不影响基本表中数据的存储状况。

4. 聚集函数

为了进一步方便用户,增强检索功能,SQL 提供了许多聚集函数,主要有:

COUNT(*)	统计元组个数
COUNT([DISTINCT ALL]<列名>)	统计一列中值的个数
SUM([DISTINCT ALL]<列名>)	计算一列值的总和(此列必须是数值型)
AVG([DISTINCT ALL]<列名>)	计算一列值的平均值(此列必须是数值型)
MAX([DISTINCT ALL]<列名>)	求一列值中的最大值
MIN([DISTINCT ALL]<列名>)	求一列值中的最小值

如果指定 DISTINCT 短语,则表示在计算时要取消指定列中的重复值。如果不指定 DISTINCT 短语或指定 ALL 短语(ALL 为默认值),则表示不取消重复值。

当遇到空值时,聚集函数中除 COUNT(*) 函数外,其他函数都跳过空值而只处理非空值。COUNT(*) 是对元组进行计数,某个元组的一个或部分列取空值不影响其统计结果。

[例 3.41] 查询学生总人数。

```
SELECT COUNT(*)
FROM Student;
```

[例 3.42] 查询选修了课程的学生人数。

```
SELECT COUNT(DISTINCT Sno)
FROM SC;
```

学生每选修一门课,在 SC 中都有一条相应的记录。一个学生可选修多门课程,为避免重

复计算学生人数，必须在 COUNT 函数中用 DISTINCT 短语。

[例 3.43] 计算选修 81001 号课程的学生平均成绩。

```
SELECT AVG(Grade)
FROM SC
WHERE Cno='81001';
```

[例 3.44] 查询选修 81001 号课程的学生最高分数。

```
SELECT MAX(Grade)
FROM SC
WHERE Cno='81001';
```

[例 3.45] 查询学号为 20180003 的学生选修课程的总学分数。

```
SELECT SUM(Ccredit)
FROM SC, Course
WHERE Sno='20180003' AND SC.Cno=Course.Cno;
```

注意：WHERE 子句不能直接用聚集函数作为条件表达式。聚集函数只能用于 SELECT 子句和 GROUP BY 子句中的 HAVING 短语。

5. GROUP BY 子句

GROUP BY 子句将查询结果按某一列或多列的值分组，值相等的为一组，如果是多列，则先对第一列的值分组，然后对每一组中的值按照第二列值分组，以此类推。结果集中如果有重复的列组值，则将其合并为一行输出。

例如，若对 A, B 两列进行如下分组：

```
(1,2)
(1,2)
(1,3)
(2,4)
```

那么，最后的分组结果为

```
(1,2)
(1,3)
(2,4)
```

对查询结果分组的目的之一是细化聚集函数的作用对象。如果未对查询结果分组，聚集函数将作用于整个查询结果，如前面的例 3.41~3.45。分组后聚集函数将作用于每一个组，即每组都有一个聚集函数值。

[例 3.46] 求各个课程号及选修该课程的人数。

```
SELECT Cno, COUNT( Sno)
FROM SC
GROUP BY Cno;
```

该语句对查询结果按 Cno 的值分组, 所有具有相同 Cno 值的元组为一组, 然后对每一组应用聚集函数 COUNT 进行计算, 以求得该组的学生人数。

查询结果可能如图 3.6 所示。

如果分组后还要求按一定的条件对这些组进行筛选, 最终只输出满足指定条件的组, 则可以使用 HAVING 短语指定筛选条件。

[例 3.47] 查询 2019 年第 2 学期选修课程数超过 10 门的学生学号。

```
SELECT Sno
FROM SC
WHERE Semester='20192'          /* 先求出 2019 年第 2 学期选课的所有学生 */
GROUP BY Sno                    /* 用 GROUP BY 子句按 Sno 进行分组 */
HAVING COUNT( *) >10;           /* 用聚集函数 COUNT 对每一组进行计数 */
```

图 3.6 例 3.46 查询结果

HAVING 短语给出了选择组的条件, 只有满足条件(即一个组中元组个数>10, 表示此学生 2019 年第 2 学期选修的课超过 10 门)的组才会被选出来。

WHERE 子句与 HAVING 短语的区别在于作用对象不同。WHERE 子句作用于基本表或视图, 从中选择满足条件的元组。HAVING 短语作用于组, 从中选择满足条件的组。

关系数据库管理系统在处理 SQL 语句时, 先处理 WHERE 子句, 根据条件选出合格元组, 生成一个临时表, 本例中选出了 2019 年第 2 学期选课的所有学生, 再使用 GROUP BY 子句对临时表按照学号分组, 最后用 HAVING 条件中的聚集函数 COUNT 对每一组进行计数, 选出 COUNT >10 的组, 输出该组的 Sno。

[例 3.48] 查询平均成绩大于或等于 90 分的学生学号和平均成绩。

```
SELECT Sno, AVG(Grade)
FROM SC
GROUP BY Sno
HAVING AVG(Grade) >= 90;
```

下面的语句是错误的:

```
SELECT Sno, AVG(Grade)
FROM SC
WHERE AVG(Grade) >= 90
GROUP BY Sno;
```

因为 WHERE 子句不能用聚集函数作为条件表达式。

6. LIMIT 子句

LIMIT 子句用于限制 SELECT 语句查询结果的(元组)数量,其一般形式如下:

LIMIT <行数 1>[OFFSET <行数 2>];

语义是取<行数 1>,忽略前<行数 2>行,作为查询结果数据。OFFSET 可以省略,代表不忽略任何行。

实际应用中 LIMIT 子句经常和 ORDER BY 子句一起使用。

[例 3.49] 查询选修数据库系统概论课程且成绩排名在前 10 名的学生的学号。

```
SELECT Sno
FROM SC, Course
WHERE Course. Cname='数据库系统概论' AND SC. Cno=Course. Cno
ORDER BY Grade DESC
LIMIT 10;           /* 取前 10 行数据为查询结果 */
```

[例 3.50] 查询平均成绩排名在第 3~7 名的学生的学号和平均成绩。

```
SELECT Sno,AVG(Grade)
FROM SC
GROUP BY Sno
ORDER BY AVG(Grade) DESC
LIMIT 5 OFFSET 2;  /* 取 5 行数据,忽略前 2 行,之后为查询结果数据 */
```

3.3.2 连接查询

前面的查询都是针对一个表进行的。若一个查询同时涉及两个以上的表,则称之为连接查询。连接查询是关系数据库中最常用的查询,包括等值连接查询、非等值连接查询、自然连接查询、自身连接查询、外连接查询、复合条件连接查询、多表连接查询等。

连接的概念在第 2 章做初步讲解,读者可以复习一下 2.4.2 小节中的有关概念。

1. 等值与非等值连接查询

连接查询的 WHERE 子句中用来连接两个表的条件称为连接条件或连接谓词,其一般格式如下:

[<表名 1>.]<列名 1><比较运算符>[<表名 2>.]<列名 2>

其中,比较运算符主要有=、>、<、>=、<=、!= (或<>)等。

此外,连接谓词还可以使用下面的形式:

[<表名 1>.]<列名 1> BETWEEN [<表名 2>.]<列名 2> AND [<表名 2>.]<列名 3>

比较运算符为“=”的连接查询称为等值连接查询。使用其他比较运算符的连接查询则称为非等值连接查询。

连接谓词中的“列名”称为连接字段。连接条件中的各连接字段类型必须是可比的，但名字不必相同。

[例 3.51] 查询每个学生及其选修课程的情况。

学生情况存放在 Student 表中，学生选课情况存放在 SC 表中，所以本查询实际上涉及 Student 与 SC 两个表。这两个表之间的联系是通过公共属性 Sno 实现的。

```
SELECT Student. *, SC. *
FROM Student, SC
WHERE Student. Sno=SC. Sno    /* 将 Student 与 SC 中同一学生的元组连接起来 */
```

假设 Student 表、SC 表的数据如第 2 章图 2.1 所示，本查询的执行结果如图 3.7 所示。

Student. Sno	Sname	Ssex	Sbirthdate	Smajor	SC.Sno	Cno	Grade	Semester	Teachingclass
20180001	李勇	男	2000-3-8	信息安全	20180001	81001	85	20192	81001-01
20180001	李勇	男	2000-3-8	信息安全	20180001	81002	96	20201	81002-01
20180001	李勇	男	2000-3-8	信息安全	20180001	81003	87	20202	81003-01
20180002	刘晨	女	1999-9-1	计算机科学与技术	20180002	81001	80	20192	81001-02
20180002	刘晨	女	1999-9-1	计算机科学与技术	20180002	81002	98	20201	81002-01
20180002	刘晨	女	1999-9-1	计算机科学与技术	20180002	81003	71	20202	81003-02
...	...	...	...	...	...	...	...	...	...

图 3.7 例 3.51 查询结果示例

说明：为节省篇幅，图 3.7 没有列出查询的全部结果，仅列出了“李勇”和“刘晨”两个学生及其选课情况。

关系数据库管理系统执行该连接操作的一种可能过程是：首先在表 Student 中找到第一个元组，然后从头开始扫描 SC 表，逐一查找与 Student 第一个元组的 Sno 相等的 SC 元组，

找到后就将 Student 中的第一个元组与该元组拼接起来, 形成结果表中的一个元组。SC 全部查找完后, 再找 Student 中第二个元组, 然后再从头开始扫描 SC, 逐一查找满足连接条件的元组, 找到后就将 Student 中的第二个元组与该元组拼接起来, 形成结果表中的一个元组。重复上述操作, 直到 Student 中的全部元组都处理完毕为止(见图 3.8 所示)。这就是嵌套循环连接算法的基本思想。



图 3.8 关系数据库管理系统执行连接操作的过程示意图

如果在 SC 表的 Sno 上建立了索引, 就不需要每次都全表扫描 SC 表了, 而是根据 Sno 值通过索引找到相应的 SC 元组。用索引查询 SC 中满足条件的元组一般会比全表扫描快。

2. 自然连接查询

把结果表目标列中重复的属性列去掉的等值连接查询则为自然连接查询, 如例 3.52 中去掉了 SC 表中的 Sno。

[例 3.52] 查询每个学生的学号、姓名、性别、出生日期、主修专业及该学生选修课程的课程号与成绩。

```
SELECT Student.Sno,Sname,Ssex,Sbirthdate,Smajor,Cno,Grade
FROM Student,SC
WHERE Student.Sno=SC.Sno;
```

本例中, 由于 Sname、Ssex、Sbirthdate、Smajor、Cno 和 Grade 属性列在 Student 表与 SC 表中是唯一的, 因此引用时可以去掉表名前缀; 而 Sno 在两个表都出现了, 因此 SELECT 子句和 WHERE 子句在引用时必须加上表名前缀。

3. 复合条件连接查询

使用一条 SQL 语句可以同时完成选择和连接查询, 这时 WHERE 子句是由连接谓词和选择谓词组成的复合条件。WHERE 子句中有多个条件的连接查询, 称为复合条件连接查询。

[例 3.53] 查询选修 81002 号课程且成绩在 90 分以上的所有学生的学号和姓名。

```
SELECT Student.Sno,Sname
```

```

FROM Student,SC
WHERE Student.Sno=SC.Sno AND          /* 连接谓词 */
      SC.Cno='81002' AND SC.Grade>90; /* 其他选择条件 */

```

该查询的一种优化(高效)的执行过程是,先从 SC 中挑选出 Cno='81002'并且 Grade>90 的元组形成一个中间关系,再和 Student 中满足连接条件的元组进行连接得到最终的结果关系。

4. 自身连接查询

连接操作不仅可以在两个表之间进行,也可以是一个表与其自身进行连接,此时的连接查询称为表的**自身连接查询**。

[例 3.54] 查询每一门课的间接先修课(即先修课的先修课)。

先来分析一下,题目要求查询每一门课程的先修课的先修课,在 Course 表中只有每门课的直接先修课的课程号 Cpno,而没有先修课的先修课课程号。要查询每一门课的间接先修课,必须先对一门课找到其直接先修课 Cpno,再按此先修课的课程号查找它的先修课程。这相当于将 Course 表与其自身连接后,取第一个副本的课程号与第二个副本的先修课号作为目标列中的属性。

具体写 SQL 语句时,为清楚起见,可以为 Course 表取两个别名:FIRST 和 SECOND,也可以在考虑问题时就把 Course 表想成是两个完全一样的表,一个是 FIRST 表,另一个是 SECOND 表,然后把这两个表连接。这就是 Course 表与其自身连接。完成该查询的 SQL 语句如下:

```

SELECT FIRST.Cno,SECOND.Cpno
FROM Course FIRST,Course SECOND
WHERE FIRST.Cpno=SECOND.Cno and SECOND.Cpno IS NOT NULL;

```

在 FROM 子句中为 Course 表定义了两个不同的别名,这样就可以分别在 SELECT 子句和 WHERE 子句中的属性名前加这两个别名来进行区分(见图 3.9 所示)。

FIRST表(Course表)

课程号 Cno	课程名 Cname	学分 Ccredit	先修课 Cpno
81001	程序设计基础与C语言	4	
81002	数据结构	4	81001
81003	数据库系统概论	4	81002
81004	信息系统概论	4	81003
81005	操作系统	4	81001
81006	Python语言	3	81002
81007	离散数学	4	
81008	大数据技术概论	4	81003

SECOND表(Course表)

课程号 Cno	课程名 Cname	学分 Ccredit	先修课 Cpno
81001	程序设计基础与C语言	4	
81002	数据结构	4	81001
81003	数据库系统概论	4	81002
81004	信息系统概论	4	81003
81005	操作系统	4	81001
81006	Python语言	3	81002
81007	离散数学	4	
81008	大数据技术概论	4	81003

图 3.9 自身连接查询示例

查询结果如图 3.10 所示。

Cno	Cpno
81003	81001
81004	81002
81006	81001
81008	81002

图 3.10 例 3.54 查询结果

注意：在 Course 表中只列出 Cno 课程的一门直接先修课 Cpno，这样才能保证 Cno 是主码。实际情况会复杂些，一门课程可能会有几门直接先修课。这时就需要重新设计 S-C-SC 模式，有关方法将在第二篇设计与应用开发篇中讲解。

5. 外连接查询

在第 2 章 2.4.2 小节也讲解过外连接的有关概念。

在通常的连接操作中，只有满足连接条件的元组才能作为结果输出。如例 3.51 的结果表中就不会有学号为 20180006 和 20180007 的两个学生的信息，原因在于他们还没有选课，在 SC 表中没有相应的元组，导致 Student 中这些元组在连接时被舍弃了（这些被舍弃的元组称为悬浮元组）。

但是，有时我们想以 Student 表为主体列出每个学生的基本情况及其选课情况，若某个学生没有选课，则只输出其基本情况的数据，而把选课信息填为空值 NULL，这时就需要使用外连接查询。

[例 3.55]

```
SELECT Student. Sno, Sname, Ssex, Sbirthdate, Smajor, Cno, Grade
FROM Student LEFT OUTER JOIN SC ON (Student. Sno=SC. Sno);
```

查询结果如图 3.11 所示。

左外连接列出 FROM 子句中左边关系（如本例 Student）所有的元组，右外连接列出 FROM 子句中右边关系中（如本例 SC）所有的元组。

6. 多表连接查询

连接查询除了可以是两表连接查询、一个表与其自身连接查询外，还可以是两个以上的表进行连接查询，后者通常称为多表连接查询。

[例 3.56] 查询每个学生的学号、姓名、选修的课程名及成绩。

本例查询涉及 3 个表，存放学生学号和姓名的 Student 表、存放学生选课成绩的 SC 表和存放课程名的 Course 表。完成该查询的 SQL 语句如下：

```
SELECT Student. Sno, Sname, Cname, Grade
FROM Student, SC, Course
WHERE Student. Sno=SC. Sno AND SC. Cno=Course. Cno;
```


Student.Sno	Sname	Ssex	Sbirthdate	Smajor	Cno	Grade
20180001	李勇	男	2000-3-8	信息安全	81001	85
20180001	李勇	男	2000-3-8	信息安全	81002	96
20180001	李勇	男	2000-3-8	信息安全	81003	87
20180002	刘晨	女	1999-9-1	计算机科学与技术	81001	80
20180002	刘晨	女	1999-9-1	计算机科学与技术	81002	98
20180002	刘晨	女	1999-9-1	计算机科学与技术	81003	71
20180003	王敏	女	2001-8-1	计算机科学与技术	81001	81
20180003	王敏	女	2001-8-1	计算机科学与技术	81002	76
20180004	张立	男	2000-1-8	计算机科学与技术	81001	56
20180004	张立	男	2000-1-8	计算机科学与技术	81002	97
20180005	陈新奇	男	2001-11-1	信息管理与信息系统	81003	68
20180006	赵明	男	2000-6-12	数据科学与大数据技术	NULL	NULL
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术	NULL	NULL

图 3.11 例 3.55 查询结果

关系数据库管理系统在执行多表连接查询时，通常是先进行两个表的连接查询，再将其结果与第三个表进行连接查询。本例的一种可能的执行方式是，先将 Student 表与 SC 表进行连接查询，得到每个学生的学号、姓名、所选课程号和相应的成绩，然后再将其与 Course 表进行连接查询，得到最终结果。

关系数据库管理系统执行连接查询时，多个表连接的次序会影响执行效率。这是关系数据库管理系统查询优化算法要考虑的问题，即通过代价估算确定实际执行连接的次序。

3.3.3 嵌套查询

在 SQL 中，一个 SELECT-FROM-WHERE 语句称为一个查询块。将一个查询块嵌套在另一个查询块的 WHERE 子句或 HAVING 短语的条件中的查询称为嵌套查询(nested query)。例如，查询选修 81003 号课程的所有学生姓名，其 SQL 语句如下：

```
SELECT Sname           /* 外层查询或父查询 */
FROM Student
WHERE Sno IN
    (SELECT Sno         /* 内层查询或子查询 */
     FROM SC
     WHERE Cno='81003');
```

本例中，下层查询块 SELECT Sno FROM SC WHERE Cno='201803'是嵌套在上层查询块 SE-

LECT Sname FROM Student 的 WHERE 条件中的。上层的查询块称为外层查询或父查询，下层查询块称为内层查询或子查询。

SQL 语言允许多层嵌套查询，即一个子查询中还可以嵌套其他子查询。需要特别指出的是，子查询的 SELECT 语句中不能使用 ORDER BY 子句，该子句只能对最终查询结果排序。

嵌套查询使得用户可以用多个简单查询构成复杂的查询，从而增强 SQL 的查询能力。以层层嵌套的方式来构造程序正是 SQL 中“结构化”的含义所在。

1. 带有 IN 谓词的子查询

在嵌套查询中，子查询的结果往往是一个集合，所以谓词 IN 是嵌套查询中最经常使用的谓词。

[例 3.57] 查询与“刘晨”在同一个主修专业的学生学号、姓名和主修专业。

先分步来完成此查询，然后再构造嵌套查询。

① 确定“刘晨”的主修专业名。

```
SELECT Smajor
FROM Student
WHERE Sname='刘晨';
```

结果为计算机科学与技术。

② 查找所有主修计算机科学与技术专业的学生。

```
SELECT Sno,Sname,Smajor
FROM Student
WHERE Smajor='计算机科学与技术';
```

查询结果如图 3.12 所示。

Sno	Sname	Smajor
20180002	刘晨	计算机科学与技术
20180003	王敏	计算机科学与技术
20180004	张立	计算机科学与技术

图 3.12 例 3.57 查询结果

分步写查询毕竟比较麻烦，上述查询可以用嵌套查询来实现，即将第一步查询嵌入第二步查询的条件，构造嵌套查询语句如下：

```
SELECT Sno,Sname,Smajor /* 例 3.57 的解法一 */
FROM Student
WHERE Smajor IN
    (SELECT Smajor
```

```
FROM Student
WHERE Sname='刘晨');
```

本例中，子查询的查询条件不依赖于父查询，称为**不相关子查询**。

关系数据库管理系统求解该查询时，实际上也是由里向外分步去做的。即先处理子查询，确定“刘晨”的主修专业，得到结果“计算机科学与技术”，子查询的结果用于建立其父查询 WHERE 子句的查找条件，得到如下的语句：

```
SELECT Sno,Sname,Smajor
FROM Student
WHERE Smajor IN ('计算机科学与技术');
```

然后执行该语句。

本例中的查询也可以用自身连接来完成：

```
SELECT S1.Sno,S1.Sname,S1.Smajor /* 例 3.57 的解法二 */
FROM Student S1,Student S2
WHERE S1.Smajor=S2.Smajor AND S2.Sname='刘晨';
```

[例 3.58] 查询选修了课程名为“信息系统概论”的学生的学号和姓名。

本查询涉及学号、姓名和课程名三个属性。学号和姓名存放在 Student 表中，课程名存放在 Course 表中，但 Student 与 Course 两个表之间没有直接联系，必须通过 SC 表建立它们之间的联系。所以本查询实际上涉及三个关系。

SELECT Sno,Sname	③ 在 Student 关系中
FROM Student	取出 Sno 和 Sname
WHERE Sno IN	
(SELECT Sno	② 在 SC 关系中找到选修
FROM SC	81004 号课程的学生学号
WHERE Cno IN	
(SELECT Cno	① 在 Course 关系中找到
FROM Course	“信息系统概论”的课程号，
WHERE Cname='信息系统概论'	结果为 81004
)	
);	

本查询同样可以用连接查询实现：

```
SELECT Student.Sno,Sname
FROM Student,SC,Course
WHERE Student.Sno=SC.Sno AND
```

```
SC. Cno=Course. Cno AND  
Course. Cname='信息系统概论';
```

有些嵌套查询可以用连接查询替代,有些是不能替代的。从例 3.57 和例 3.58 可以看到,查询涉及多个关系时,用嵌套查询逐步求解层次清楚,易于构造,具有结构化程序设计的优点。但是,有的查询使用连接查询可能效率更高些。所以在实际应用中,需要根据关系数据库管理系统的查询处理算法效率来选择使用哪种 SQL 语句。

2. 带有比较运算符的子查询

带有比较运算符的子查询是指父查询与子查询之间用比较运算符进行连接。当用户能确切知道内层查询返回的是单个值时,可以用>、<、=、>=、<=、!=或<>等比较运算符。

例如在例 3.57 中,由于一个学生只可能主修一个专业,也就是说内查询的结果是一个值,因此可以用“=”代替“IN”:

```
SELECT Sno,Sname,Smajor          /* 例 3.57 的解法三 */  
FROM Student  
WHERE Smajor =  
      (SELECT Smajor  
       FROM Student  
       WHERE Sname='刘晨');
```

实现同一个查询请求可以有多种方法,就如上面的例 3.57 分别给出了三种不同的解法。不同的 SQL 语句执行效率会有差别,甚至差别会很大。这就是数据库编程人员应该掌握数据库性能调优技术的原因。

例 3.57 和例 3.58 中子查询的查询条件不依赖于父查询,这类子查询称为不相关子查询。不相关子查询是较简单的一类子查询。如果子查询的查询条件依赖于父查询,这类子查询称为相关子查询 (correlated subquery), 整个查询语句称为相关嵌套查询 (correlated nested query) 语句。

例 3.59 就是一个相关子查询的例子。

[例 3.59] 找出每个学生超过他自己选修课程平均成绩的课程号。

```
SELECT Sno,Cno  
FROM SC x  
WHERE Grade >=(SELECT AVG(Grade)          /* 某学生的平均成绩 */  
               FROM SC y  
               WHERE y. Sno=x. Sno);
```

x、y 分别是表 SC 的别名,又称为元组变量,可以用来表示 SC 的一个元组。内层查询是求一个学生所有选修课程平均成绩的,至于是哪个学生的平均成绩要看参数 x. Sno 的值,而该值是与父查询相关的,因此这类查询称为相关子查询。

该语句的一种可能的执行过程采用以下三个步骤。

- ① 从外层查询中取出 SC 的一个元组 x, 将元组 x 的 Sno 值(20180001)传送给内层查询。

```
SELECT AVG(Grade)
FROM SC y
WHERE y.Sno='20180001';
```

- ② 执行内层查询, 得到值 89.3(近似值), 用该值代替内层查询, 得到外层查询:

```
SELECT Sno, Cno
FROM SC x
WHERE Grade >= 89.3;
```

- ③ 执行这个查询, 得到(20180001, 81002)

然后外层查询取出下一个元组重复做上述步骤①至③的处理, 直到外层的 SC 元组全部处理完毕。结果如下:

```
(20180001,81002)
(20180002,81002)
(20180003,81001)
(20180004,81003)
(20180005,81003)
```

求解相关子查询不能像求解不相关子查询那样, 先一次将子查询求解出来, 然后求解父查询, 由于内层查询与外层查询有关, 因此必须反复求值。

3. 带有 ANY(SOME) 或 ALL 谓词的子查询

子查询返回单值时可以用比较运算符, 但返回多值时要用 ANY(有的系统用 SOME)或 ALL 谓词修饰符。而使用 ANY 或 ALL 谓词时必须同时使用比较运算符。其语义如下所示:

> ANY	大于子查询结果中的某个值
> ALL	大于子查询结果中的所有值
< ANY	小于子查询结果中的某个值
< ALL	小于子查询结果中的所有值
>= ANY	大于或等于子查询结果中的某个值
>= ALL	大于或等于子查询结果中的所有值
<= ANY	小于或等于子查询结果中的某个值
<= ALL	小于或等于子查询结果中的所有值
= ANY	等于子查询结果中的某个值
= ALL	等于子查询结果中的所有值(通常没有实际意义)
!=(或<>) ANY	不等于子查询结果中的某个值
!=(或<>) ALL	不等于子查询结果中的任何一个值

[例 3.60] 查询非计算机科学与技术专业中比计算机科学与技术专业任意一个年龄小(出生日期晚)的学生的姓名、出生日期和主修专业。

```
SELECT Sname, Sbirthdate, Smajor
FROM Student
WHERE Sbirthdate > ANY (SELECT Sbirthdate
                        FROM Student
                        WHERE Smajor = '计算机科学与技术')
AND Smajor <> '计算机科学与技术';      /* 注意这是父查询块中的条件 */
```

查询结果如图 3.13 所示。

Sname	Sbirthdate	Smajor
李勇	2000-3-8	信息安全
陈新奇	2001-11-1	信息管理与信息系统
赵明	2000-6-12	数据科学与大数据技术
王佳佳	2001-12-7	数据科学与大数据技术

图 3.13 例 3.60 查询结果

关系数据库管理系统执行此查询时,首先处理子查询,找出计算机科学与技术专业中所有学生的出生日期,构成一个集合(1999-9-1, 2001-8-1, 2000-1-8)。然后处理父查询,找所有不是计算机科学与技术专业且出生日期大于集合中任意一个的学生。

本查询也可以用聚集函数来实现,首先用子查询找出计算机科学与技术专业学生中最早出生的日期(1999-9-1),然后在父查询中查所有非计算机科学与技术专业且出生日期比 1999-9-1 晚的学生。SQL 语句如下:

```
SELECT Sname, Sbirthdate, Smajor
FROM Student
WHERE Sbirthdate >
      (SELECT MIN(Sbirthdate)
       FROM Student
       WHERE Smajor = '计算机科学与技术')
AND Smajor <> '计算机科学与技术';
```

[例 3.61] 查询非计算机科学与技术专业中比计算机科学与技术专业所有学生年龄都小(出生日期晚)的学生的姓名及出生日期。

SQL 语句如下:

```
SELECT Sname, Sbirthdate
FROM Student
```

```

WHERE Sbirthdate > ALL
    (SELECT Sbirthdate
     FROM Student
     WHERE Smajor='计算机科学与技术')
AND Smajor < > '计算机科学与技术';

```

关系数据库管理系统执行此查询时，首先处理子查询，找出计算机科学与技术专业中所有学生的年龄，构成一个集合(1999-9-1, 2001-8-1, 2000-1-8)。然后处理父查询，找所有不是计算机科学与技术专业且出生日期晚于集合中所有值的学生。

查询结果如图 3.14 所示。

name	birthdate
陈新奇	2001-11-1
王佳佳	2001-12-7

图 3.14 例 3.61 查询结果

本查询同样也可以用聚集函数实现。SQL 语句如下：

```

SELECT Sname, Sbirthdate
FROM Student
WHERE Sbirthdate >
    (SELECT MAX(Sbirthdate) /* 计算机科学与技术专业所有学生中最晚出生日期 */
     FROM Student
     WHERE Smajor='计算机科学与技术')
AND Smajor < > '计算机科学与技术';

```

关系数据库管理系统执行此查询时，首先处理子查询，找出计算机科学与技术专业所有学生中最晚的出生日期，结果是 2001-8-1，然后处理父查询，找所有不是计算机科学与技术专业且出生日期晚于 2001-8-1 的学生。

用聚集函数实现子查询通常比直接用 ANY 或 ALL 查询效率要高。ANY、ALL 与聚集函数的对应关系如表 3.6 所示。

表 3.6 ANY(或 SOME)、ALL 谓词与聚集函数、IN 谓词的等价转换关系

谓词	=	< >或!=	<	<=	>	>=
ANY	IN	--	<MAX	<=MAX	>MIN	>=MIN
ALL	--	NOT IN	<MIN	<=MIN	>MAX	>=MAX

表 3.6 中，=ANY 等价于 IN 谓词，<ANY 等价于 <MAX，<>ALL 等价于 NOT IN 谓词，<ALL 等价于 <MIN，等等。

4. 带有 EXISTS 谓词的子查询

EXISTS 代表存在量词 $\exists$ 。带有 EXISTS 谓词的子查询不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”。

可以利用 EXISTS 来判断 $x \in S$ 、 $S \subseteq R$ 、 $S=R$ 、 $S \cap R$ 非空等是否成立。

[例 3.62] 查询所有选修了 81001 号课程的学生姓名。

本查询涉及 Student 和 SC 表。可以在 Student 中依次取每个元组的 Sno 值，用此值去检查 SC 表。若 SC 中存在这样的元组，其 Sno 值等于此 Student. Sno 值，并且其 Cno='81001'，则取此 Student. Sname 送入结果表。将此想法写成 SQL 语句如下：

```
SELECT Sname
FROM Student
WHERE EXISTS
    (SELECT *
     FROM SC
     WHERE Sno=Student. Sno AND Cno='81001');
```

使用存在量词 EXISTS 后，若内层查询结果非空，则外层的 WHERE 子句返回真值，否则返回假值。

由 EXISTS 引出的子查询，其目标列表表达式通常都用 *，因为带 EXISTS 的子查询只返回真值或假值，给出列名无实际意义。

本例中子查询的查询条件依赖于外层父查询的某个属性值(Student 的 Sno 值)，因此也是相关子查询。这个相关子查询的处理过程是：首先取外层查询中 Student 表的第一个元组，根据它与内层查询相关的属性值(Sno 值)处理内层查询，若 WHERE 子句返回值为真，则取外层查询中该元组的 Sname 放入结果表，然后再取 Student 表的下一个元组。重复这一过程，直至外层 Student 表全部检查完为止。

本例中的查询也可以用连接运算来实现，读者可以参照有关的例子自己给出相应的 SQL 语句。

与 EXISTS 谓词相对应的是 NOT EXISTS 谓词。使用存在量词 NOT EXISTS 后，若内层查询结果为空，则外层的 WHERE 子句返回真值，否则返回假值。

[例 3.63] 查询没有选修 81001 号课程的学生姓名。

```
SELECT Sname
FROM Student
WHERE NOT EXISTS
    (SELECT *
     FROM SC
     WHERE Sno=Student. Sno AND Cno='81001');
```


一些带 EXISTS 或 NOT EXISTS 谓词的子查询不能被其他形式的子查询等价替换,但所有带 IN 谓词、比较运算符、ANY 和 ALL 谓词的子查询都能用带 EXISTS 谓词的子查询等价替换。例如带有 IN 谓词的例 3.57 查询与“刘晨”在同一个主修专业的学生。可以用如下带 EXISTS 谓词的子查询替换:

```
SELECT Sno,Sname,Smajor                                /* 例 3.57 的解法 4 */
FROM Student S1
WHERE EXISTS
    (SELECT *
     FROM Student S2
     WHERE S2.Smajor=S1.Smajor AND S2.Sname='刘晨');
```

由于带 EXISTS 量词的相关子查询只关心内层查询是否有返回值,并不需要查具体值,因此其效率并不一定低于不相关子查询,有时是高效的方法。

[例 3.64] 查询选修了全部课程的学生姓名。

SQL 中没有全称量词(for all),但是可以把带有全称量词的谓词转换为等价的带有存在量词的谓词:

$$(\forall x)P \equiv \neg(\exists x(\neg P))$$

由于没有全称量词,可将题目的意思转换成等价的用存在量词的形式:查询这样的学生,没有一门课程是他不选修的。其 SQL 语句如下:

```
SELECT Sname
FROM Student
WHERE NOT EXISTS
    (SELECT *
     FROM Course
     WHERE NOT EXISTS
        (SELECT *
         FROM SC
         WHERE Sno=Student.Sno AND Cno=Course.Cno));
```

从而用 EXIST/NOT EXIST 来实现带全称量词的查询。

[例 3.65] 查询至少选修了学生 20180002 选修的全部课程的学生学号。

本查询可以用逻辑蕴涵来表达:查询学号为 x 的学生,对所有的课程 y,只要 20180002 学生选修了课程 y,则 x 也选修了 y。形式化表示如下:

用 p 表示谓词“学生 20180002 选修了课程 y”

用 q 表示谓词“学生 x 选修了课程 y”

则上述查询为

$$(\forall y)p \rightarrow q$$

SQL 语言中没有蕴涵(implication)逻辑运算,但是可以利用谓词演算将一个逻辑蕴涵的谓词等价转换为

$$p \rightarrow q \equiv \neg p \vee q$$

该查询可以转换为如下等价形式:

$$(\forall y) p \rightarrow q \equiv \neg(\exists y(\neg(p \rightarrow q))) \equiv \neg(\exists y(\neg(\neg p \vee q))) \equiv \neg \exists y(p \wedge \neg q)$$

它所表达的语义为:不存在这样的课程 y, 学生 20180002 选修了 y, 而学生 x 没有选修。用 SQL 语言表示如下:

```
SELECT Sno
FROM Student
WHERE NOT EXISTS
    (SELECT *          /* 这是一个相关子查询 */
     FROM SC SCX        /* 父查询和子查询均引用了 SC 表 */
     WHERE SCX.Sno='20180002' AND
          NOT EXISTS
            (SELECT *
             FROM SC SCY    /* 用别名 SCX,SCY 将父查询 */
             WHERE SCY.Sno=Student.Sno AND /* 与子查询中的 SC 表区分开 */
                   SCY.Cno=SCX.Cno));
```

3.3.4 集合查询

SELECT 语句的查询结果是元组的集合,所以对多个 SELECT 语句的结果可进行集合操作。集合操作主要包括并操作 UNION、交操作 INTERSECT 和差操作 EXCEPT。

注意:参加集合操作的各查询结果的列数必须相同,对应项的数据类型也必须相同。

[例 3.66] 查询计算机科学与技术专业的学生及年龄不大于 19 岁(包括等于 19 岁)的学生。

```
SELECT *
FROM Student
WHERE Smajor='计算机科学与技术'
UNION
SELECT * FROM Student
WHERE (extract(year from current_date) - extract(year from Shirthdate)) <= 19;
```

本查询实际上是求计算机科学系的所有学生与年龄不大于 19 岁的学生的并集。使用 UNION 将多个查询结果合并起来时,系统会自动去掉重复元组。如果要保留重复元组则用 UNION ALL 操作符。

[例 3.67] 查询 2020 年第 2 学期选修了课程 81001 或 81002 的学生。

本例即查询 2020 年第 2 学期选修课程 81001 的学生集合和选修课程 81002 的学生集合的

并集。

```
SELECT Sno
FROM SC
WHERE Semester='20202' AND Cno='81001'
UNION
SELECT Sno
FROM SC
WHERE Semester='20202' AND Cno='81002';
```

[例 3. 68] 查询计算机科学与技术专业的学生与年龄不大于 19 岁的学生的交集。

```
SELECT *
FROM Student
WHERE Smajor='计算机科学与技术'
INTERSECT
SELECT *
FROM Student
WHERE( extract( year from current_date )-extract( year from Sbirthdate ) ) <= 19;
```

这实际上就是查询计算机科学与技术专业中年龄不大于 19 岁的学生。

```
SELECT *
FROM Student
WHERE Smajor='计算机科学与技术' AND
      ( extract( year from current_date )-extract( year from Sbirthdate ) ) <= 19;
```

[例 3. 69] 查询既选修了课程 81001 又选修了课程 81002 的学生。就是查询选修课程 81001 的学生集合与选修课程 81002 的学生集合的交集。

```
SELECT Sno
FROM SC
WHERE Cno='81001'
INTERSECT
SELECT Sno
FROM SC
WHERE Cno='81002';
```

本例也可以表示为

```
SELECT Sno
FROM SC
WHERE Cno='81001' AND Sno IN
```

```
(SELECT Sno  
FROM SC  
WHERE Cno='81002');
```

[例 3.70] 查询计算机科学与技术专业的学生与年龄不大于 19 岁的学生的差集。

```
SELECT *  
FROM Student  
WHERE Smajor='计算机科学与技术'  
EXCEPT  
SELECT *  
FROM Student  
WHERE (extract(year from current_date)-extract(year from Sbirthdate)) <= 19;
```

也就是查询计算机科学与技术专业中年龄大于 19 岁的学生。

```
SELECT *  
FROM Student  
WHERE Smajor='计算机科学与技术' AND  
      (extract(year from current_date)-extract(year from Sbirthdate)) > 19;
```

3.3.5 基于派生表的查询

子查询不仅可以出现在 WHERE 子句中, 还可以出现在 FROM 子句中, 这时子查询生成的临时派生表 (derived table) 成为主查询的查询对象。例如, 例 3.59 也可以用如下的查询完成:

```
/* 找出每个学生超过他自己选修课程平均成绩的课程号 */  
SELECT Sno, Cno  
FROM SC, (SELECT Sno, Avg(Grade) FROM SC GROUP BY Sno)  
        AS Avg_SC(Avg_sno, Avg_grade)  
WHERE SC.Sno = Avg_SC.Avg_sno AND SC.Grade >= Avg_SC.Avg_grade;
```

这里 FROM 子句中的子查询将生成一个派生表 Avg_SC。该表由 Avg_sno 和 Avg_grade 两个属性组成, 记录了每个学生的学号及平均成绩。主查询将 SC 表与派生表 Avg_SC 按学号相等进行连接, 选出修课成绩大于其平均成绩的课程号。

如果子查询中没有聚集函数, 派生表可以不指定属性列, 子查询 SELECT 子句后面的列名为其默认属性。例如例 3.62 可以用如下查询完成:

```
/* 查询所有选修了 81001 号课程的学生姓名 */  
SELECT Sname  
FROM Student, (SELECT Sno FROM SC WHERE Cno='81001') AS SC1  
WHERE Student.Sno=SC1.Sno;
```

需要说明的是,通过 FROM 子句生成派生表时,AS 关键字可以省略,但必须为派生关系指定一个别名;而对于基本表,别名是可选择项。派生表是一个中间结果表,查询完成后派生表将被系统自动清除。

SELECT 语句是 SQL 语言的核心语句,从前面的例子可以看到其语句成分丰富多样。本书总结了 SELECT 语句的一般格式,读者可参见二维码内容。



扩展阅读: SELECT
语句的一般格式

3.4 数据更新

SQL 的数据更新包括插入数据、修改数据和删除数据三类语句。

3.4.1 插入数据

插入数据语句 INSERT 通常有两种形式,一种是插入一个元组,另一种是插入子查询结果。后者可以一次插入多个元组。

1. 插入一个元组

插入一个元组的 INSERT 语句的语法如下:

```
INSERT INTO <表名>[( <属性列 1>[, <属性列 2>]...)]  
VALUES (<常量 1>[, <常量 2>]...);
```

其功能是将一个新元组插入指定表中。新元组的属性列 1 的值为常量 1,属性列 2 的值为常量 2……INTO 子句中没有出现的属性列,新元组在这些列上将取空值。如果定义表时指定了相应属性列的默认值,则新元组在这些列上将取默认值。但必须注意的是,在表定义时说明了 NOT NULL 的属性列不能取空值,否则会出错。

如果 INTO 子句中没有指明任何属性列名,则新插入的元组必须在每个属性列上均有指定值。

[例 3.71] 将一个新学生元组(学号: 20180009, 姓名: 陈冬, 性别: 男, 出生日期: 2000-5-22, 主修专业: 信息管理与信息系统)插入 Student 表。

```
INSERT INTO Student (Sno, Sname, Ssex, Smajor, Sbirthdate)  
VALUES ('20180009', '陈冬', '男', '信息管理与信息系统', '2000-5-22');
```

在 INTO 子句中指出了表名 Student, 并指出了新增加的元组在哪些属性上要赋值, 属性的顺序可以与 CREATE TABLE 中的顺序不一样, 就如本例中 Sbirthdate 和 Smajor 次序交换了。VALUES 子句按照 INTO 子句指定的属性次序对新元组的各属性赋值。字符串常数要用单引号(英文符号)括起来。

[例 3.72] 将学生张成民的信息插入 Student 表。

```
INSERT INTO Student
```